

A Faster FPRAS for #Knapsack, Machine-Checked

A memo on the result, the idea, and the Lean development

Liran Markin

August 2026

Abstract

We show that the sampling stage of the current fastest approximation scheme for counting knapsack solutions – the $\tilde{O}(n^{1.5}\varepsilon^{-2})$ FPRAS of Feng and Jin (SODA 2025) – can be reimplemented so that the total running time drops to $\tilde{O}\left(n^{1.5} + \min(n\sqrt{\ell}, n^2/\ell) \cdot \varepsilon^{-2}\right) \leq \tilde{O}(n^{1.5} + n^{4/3}\varepsilon^{-2})$: never worse, and strictly faster for every $\varepsilon = o(1)$. The key observation is that the arrays their construction already stores are *exactly* a product-form factorization of the distribution being sampled, so sampling can follow that factorization instead of scanning arrays. Every component this work introduces is machine-checked in Lean 4 (~ 70 theorems, three standard axioms, rebuilt in CI); we rely on the Feng–Jin paper only for their published pipeline structure and their Theorem 6.1 convolution subroutine.

1 The problem and the result

#Knapsack asks: given items with weights $W_1, \dots, W_n$ and a capacity T , how many of the 2^n subsets have total weight at most T ? The problem is #P-hard, so one settles for a $(1 \pm \varepsilon)$ -approximation. The randomized state of the art is the FPRAS of Feng and Jin [1], running in $\tilde{O}(n^{1.5}\varepsilon^{-2})$.

Theorem 1 (main, candidate). *The Feng–Jin FPRAS can be implemented in time*

$$\tilde{O}\left(n^{1.5} + \min\left(n\sqrt{\ell}, \frac{n^2}{\ell}\right) \cdot \varepsilon^{-2}\right) \leq \tilde{O}\left(n^{1.5} + n^{4/3}\varepsilon^{-2}\right),$$

where $\ell \in [2, 8n]$ is the instance’s popular-class parameter ([1], Lemma 3.1). In the worst case over ℓ this is $\tilde{O}(n^{1.5} + n^{4/3}\varepsilon^{-2})$: equal to $\tilde{O}(n^{1.5}\varepsilon^{-2})$ at $\varepsilon = \Theta(1)$ and strictly below it for every $\varepsilon = o(1)$.

For scale: at $\varepsilon = 1/\sqrt{n}$, their bound reads $n^{2.5}$ and ours $n^{11/6}$.

2 The idea in one page

Feng–Jin build, for each node of a divide-and-conquer tree over the items, an array approximating the counting function of the node’s sub-instance. To *sample* a uniform random solution they walk this tree: at each node, the remaining weight must be split between the two children, and the split is drawn by scanning the stored arrays – length $\tilde{O}(\ell)$ per node. That scan is the entire ε^{-2} bottleneck: $\tilde{O}(n/(\ell\varepsilon^2))$ samples, each costing $\tilde{O}(\ell\sqrt{m})$.

The observation that unlocks the improvement is about their *representation*. To multiply arrays whose entries are astronomically large counts, they store every entry as $D^{\text{level}} \cdot \text{residual}$ with the residual in $[1, D)$, $\log D = \text{polylog}$, and compute merges by a $(\max, +)$ -convolution with *witness counts* (their Theorem 6.1). Unwinding the definitions, the stored value at position s equals – exactly, not approximately –

$$h(s) = \sum_{\substack{y+z=s \\ \text{level-sum attaining}}} f_L(y) f_R(z),$$

the mass of the level-sum-attaining diagonal pairs of the children’s arrays. In other words: **the arrays already store a product-form factorization of the very distribution the sampler needs**. A sampler that draws each split proportionally to that factorization introduces *zero* additional error, and their entire correctness analysis applies verbatim.

Three consequences make the splits cheap:

1. **Capacity only at the root.** Sampling the factorization means one prefix-restricted draw of the total weight at the root; every recursive step below receives an *exact* child weight. Grid re-rounding between levels has contiguous preimages, so the de-rounding draw is exact as well. No differences of approximate quantities appear anywhere.
2. **Levels, not entries.** The attaining pairs organize into *level rectangles*, and there are only $M = \tilde{O}(\text{subtree items/polylog})$ of them – far fewer than the array length near the top of the tree, which is where the scans were expensive. Within a rectangle, residuals span a polylog range, so a factor-2 class partition plus precomputed indicator convolutions (all shifts at once, by FFT, inside the construction budget) supports an exact draw with at most 4 expected rejections.
3. **Pruning.** A sample that selects k items only needs the subtree paths of those k items: skipping provably-empty children is distribution-exact, and the visit count is $\tilde{O}(k)$.

3 The cost ledger

Per sample, at depth h of a class tree: $\min(2^h, k)$ visited nodes, each costing $\min(\text{items}/2^h, \ell/2^{h/2})$ – the level count capped by the array scan. Summing the two modes:

$$\text{per sample} = \tilde{O}(\min(n_m, \ell\sqrt{k})), \quad N = \tilde{O}\left(\frac{n}{\ell\epsilon^2}\right) \implies N \cdot \text{per sample} = \tilde{O}\left(\min\left(\frac{n^2}{\ell}, n\sqrt{\ell}\right)\epsilon^{-2}\right),$$

maximized at $\ell = n^{2/3}$, giving $n^{4/3}\epsilon^{-2}$. The construction is unchanged at $\tilde{O}(n^{1.5})$; its tree accounting (every level costs $O(L\sqrt{M})$ oracle units) is also verified.

4 What is machine-checked

The Lean development (directory `lean/`, built by CI from scratch) is split into two folders that mirror the intellectual boundary:

New/	Everything this work presents. Highlights: the witness factorization and domination cores; the exact-total sampler proven <i>exactly</i> uniform, with pruning proven distribution-identical; the damage-control theorem for approximate kernels; the finite Chebyshev estimator with robustness; the composed assembly theorem; the ArraysOK interface and merge induction step, with a verified (slow) oracle inhabiting the Theorem-6.1 spec; both cost ledgers and the collapse $\min(n\sqrt{\ell}, n^2/\ell)^3 \leq n^4$; the de-rounding, class rejection, correlation-count and level-count mechanisms. The single headline statement is fprasSharp : correctness (relative- ε except probability $\leq 1/4$) $\wedge$ the full ledger $\wedge$ the $n^{4/3}$ collapse.
FengJin/	What we rely on from [1]. Their §3 structural lemmas (3.1–3.4, including both degree claims of 3.4 and a <i>constructive</i> replacement of their probabilistic hitting set) are formalized here so our sample-complexity claim rests on verified combinatorics. Their Theorem 6.1 convolution subroutine appears as an interface (Oracle.lean); its fast implementation (FFT + random primes, [1, 2]) is the one component used as published – its <i>correctness</i> spec is inhabited by our verified slow implementation, so only its speed is taken on faith.

Two further items live in prose rather than Lean, both deliberate: the routine composition of our modified sampler with their outer two-phase pipeline (weight classes, tiny items), argued in `docs/witness-sampler.md`; and the standard median amplification from success $3/4$ to $1 - \delta$.

5 Why this is the right point to stop verifying

Every mechanism introduced by this work is machine-checked; the sole unverified computational ingredient is a published subroutine of the prior state of the art, used unchanged, with its interface stated formally and inhabited by verified code. Verifying its internals (a cost-modeled FFT and the BDP24 segment machinery) is a formalization project of independent scope that would add nothing to the claims specific to this work.

References

- [1] W. Feng, C. Jin. *A Faster Algorithm for Counting Knapsack Solutions*. SODA 2025. arXiv:2410.22267.
- [2] K. Bringmann, A. Dürr, A. Polak. *Even Faster Knapsack via Bounded Monotone Min-Plus-Convolution*. 2024.
- [3] T. Chan, Q. Duan, H. Xu, Y. Zhang. *Bounded monotone min-plus-convolution*. 2022.
- [4] M. Dyer. *Approximate counting by dynamic programming*. STOC 2003.
- [5] P. Gawrychowski, L. Markin, O. Weimann. *A Faster FPTAS for #Knapsack*. ICALP 2018.